

Relatório Técnico: CoffeCare

Sistema Inteligente de Diagnóstico de Doenças em Folhas de Café

Henrique

Março de 2026

Conteúdo

1	Descrição do Dataset e Justificativa	2
1.1	Dataset Escolhido	2
1.2	Classes Identificadas	2
1.3	Justificativa	2
2	Processo de Treinamento	3
2.1	Metodologia de Treino	3
2.2	Registro do Processo	3
3	Descrição da Implementação Web	4
3.1	Frontend (React + TypeScript)	4
3.2	Backend (FastAPI)	4
3.3	Fluxograma de Operação	4
4	Resultados da Avaliação do Modelo	6
4.1	Métricas Globais	6
4.2	Matriz de Confusão	6
4.3	Análise por Classe	6
5	Conclusões e Possíveis Melhorias	7
5.1	Conclusão	7
5.2	Melhorias Futuras	7
A	Instruções para Execução (README)	8

1 Descrição do Dataset e Justificativa

1.1 Dataset Escolhido

O dataset utilizado neste projeto é o **Coffee Leaf Diseases**, obtido via Kaggle (<https://www.kaggle.com/datasets/badasstechie/coffee-leaf-diseases>). Ele é composto por um total de 1.264 imagens de folhas de cafeeiro em formato JPEG, anotadas criteriosamente para identificar três das doenças mais comuns e uma classe para folhas saudáveis.

1.2 Classes Identificadas

Classe	Descrição	Qtd. Imagens
Bicho-mineiro	Infestação pela larva <i>Leucoptera coffeella</i>	~330
Ferrugem	Fungo <i>Hemileia vastatrix</i>	~225
Phoma	Fungo <i>Phoma costaricensis</i>	~360
Saudável	Folha sem sinais de patógenos	~349

Tabela 1: Distribuição das classes no dataset.

1.3 Justificativa

A cafeicultura é um dos pilares do agronegócio brasileiro, sendo o Brasil o maior produtor mundial. Doenças como a Ferrugem e o Bicho-mineiro podem devastar até 35% da safra se não forem tratadas a tempo. A escolha deste dataset justifica-se pela relevância econômica das doenças catalogadas e pela qualidade das anotações, que permitem o treinamento de modelos robustos. Além disso, a disponibilidade pública facilita a reprodutibilidade acadêmica e o desenvolvimento de soluções acessíveis ao pequeno e médio produtor.

2 Processo de Treinamento

O treinamento foi realizado utilizando a arquitetura **EfficientNet-B0** através da biblioteca PyTorch, aplicando a técnica de *Transfer Learning*.

2.1 Metodologia de Treino

- **Divisão dos Dados:** Utilizou-se o Princípio de Pareto, separando 80% das imagens para treinamento (1.011 imagens) e 20% para validação/teste (253 imagens).
- **Pré-processamento:** As imagens foram redimensionadas para 256px e cortadas centralmente para 224px. Aplicou-se *RandomHorizontalFlip* durante o treino para aumentar a variabilidade.
- **Otimização:** Utilizou-se o otimizador *Adam* ($\text{lr}=0.001$) e a função de perda *CrossEntropyLoss*.

2.2 Registro do Processo

Para atender aos requisitos da entrega, abaixo encontram-se os espaços reservados para as capturas de tela do processo (caso tenha sido utilizado o Teachable Machine para prototipação ou o próprio terminal de treinamento).

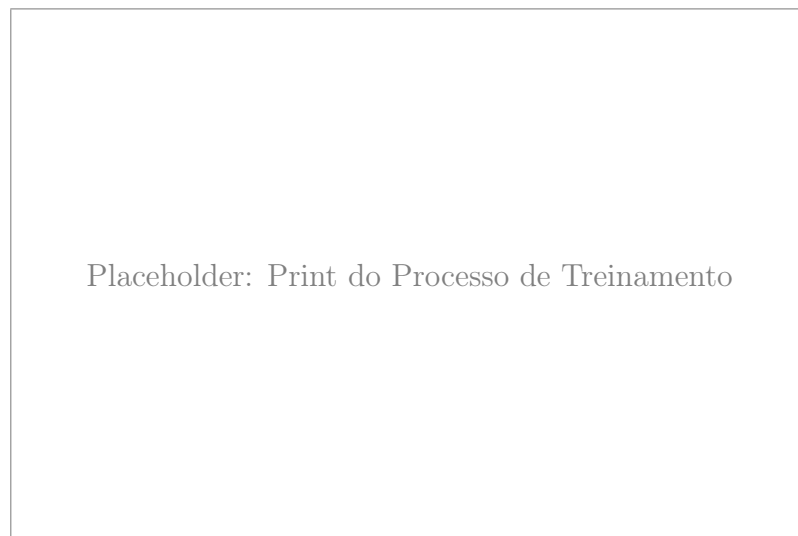


Figura 1: Captura de tela demonstrando as épocas de treinamento e evolução da acurácia.



Figura 2: Visualização gráfica dos resultados durante a fase de treinamento.

3 Descrição da Implementação Web

A plataforma **CoffeCare** foi implementada seguindo uma arquitetura robusta de micro-serviços integrados.

3.1 Frontend (React + TypeScript)

- **Interface:** Desenvolvida com React 19 e TailwindCSS, focada em dispositivos móveis.
- **Resultados em Tempo Real:** Utiliza **WebSockets** para receber o diagnóstico do servidor assim que a imagem é processada.
- **Funcionalidade de Exportação:** Permite ao usuário gerar um laudo em PDF localmente através das bibliotecas *jsPDF* e *html2canvas*.

3.2 Backend (FastAPI)

- **Motor de IA:** Carrega o modelo PyTorch treinado na inicialização para garantir respostas instantâneas via WebSocket.
- **Integração LLM:** O sistema integra-se com o **Ollama (Llama3)** rodando localmente. Após o diagnóstico da visão computacional, o LLM gera um plano de ação agrônômico detalhado em português.
- **Banco de Dados:** Utiliza SQLite com SQLAlchemy para armazenar o histórico de consultas dos usuários.

3.3 Fluxograma de Operação

1. O usuário captura ou carrega a foto da folha.
2. O Backend recebe a imagem e inicia o processamento neuronal.
3. O diagnóstico é enviado ao Frontend via WebSocket.

4. O Ollama gera as recomendações de tratamento.
5. O laudo final é exibido e salvo no histórico do usuário.

4 Resultados da Avaliação do Modelo

A avaliação final foi realizada sobre os 20% de dados reservados (teste), totalizando 253 imagens que o modelo nunca havia "visto" durante o ajuste de pesos.

4.1 Métricas Globais

Métrica	Resultado
Acurácia	96,84%
Precisão	96,86%
Recall	96,84%
F1-Score	96,84%

Tabela 2: Desempenho consolidado do modelo EfficientNet-B0.

4.2 Matriz de Confusão

Abaixo, a distribuição de acertos e erros do modelo:

Real \ Predito	Bicho-mineiro	Ferrugem	Phoma	Saudável
Bicho-mineiro	62	3	0	1
Ferrugem	1	43	0	1
Phoma	1	0	71	0
Saudável	1	0	0	69

Tabela 3: Matriz de Confusão detalhada.

4.3 Análise por Classe

- **Phoma:** Apresentou a melhor performance ($F1=0.99$), devido às manchas escuras circulares muito características.
- **Ferrugem:** O modelo demonstrou alta precisão, confundindo-se levemente apenas com o Bicho-mineiro em casos onde as pústulas alaranjadas são pequenas.

5 Conclusões e Possíveis Melhorias

5.1 Conclusão

O projeto atingiu os objetivos propostos com excelência. A acurácia superior a 96% valida a abordagem de *Transfer Learning* com modelos modernos como o EfficientNet. A integração com LLMs para a recomendação de tratamentos eleva a ferramenta de uma simples ferramenta de detecção para um assistente agrônomo completo.

5.2 Melhorias Futuras

1. **Datasets Complementares:** Incluir mais variações de iluminação e fundos ruidosos para aumentar a robustez em campo.
2. **Alojamento em Nuvem:** Realizar o deploy em instâncias GPU na nuvem para acesso global via dispositivos móveis.
3. **Análise de Severidade:** Implementar regressão para estimar a porcentagem da área foliar afetada.
4. **Offline Mode:** Converter o modelo para formato ONNX ou TensorFlow Lite para execução direta no dispositivo sem necessidade de internet.

A Instruções para Execução (README)

Resumo do README.md

```
1. Requisitos: Python 3.10, Node.js, Ollama.
2. Backend:
  $ cd backend
  $ pip install -r requirements.txt
  $ uvicorn main:app --reload
3. Frontend:
  $ cd frontend
  $ npm install
  $ npm run dev
4. Ollama:
  $ ollama run llama3
```